

Today 11:21 AM

**README(20260930-0...**
File**main.py**
Python**button.py**
Python**game_engine.py**
Python

I am completing a Vibe Coding lab. I have provided the complete repository files and README.

For this prompt, work ONLY on Task 1. Do not implement any of the other tasks or add unrelated features.

Task 1:

Fix the inverted outcome evaluation bug in `game_engine.determine_winner()`.

The intended rules are:

- ROCK beats SCISSORS
- SCISSORS beats PAPER
- PAPER beats ROCK
- Identical choices result in a DRAW

Currently, the outcome lookup dictionary has the winning and losing combinations inverted.

Please:

1. Inspect all the provided files and understand how the existing game works before making changes.
2. Fix only the incorrect outcome mappings in `determine_winner()`.
3. Preserve the existing architecture, UI, scoring behavior, CPU behavior, and timing behavior.
4. Make the smallest clean change necessary.
5. Do not implement Task 2 (First to X Wins), Task 3 (Adaptive AI), or Task 4 (Animated Icons).
6. Do not rewrite unrelated parts of the project.
7. Make sure the corrected logic works for all ROCK,

PAPER, and SCISSORS combinations, including draws.

8. Provide the exact updated code for every file that needs to change.

9. Clearly explain what you changed and why.

10. Give me a short set of manual test cases I can run locally to verify Task 1.

Do not make any unrelated changes.

Show less ^

I inspected the **README**, `main.py`, `button.py`, and `game_engine.py` before making the change. The project architecture is already correctly separated: `main.py` runs the game loop, `button.py` handles the choice buttons, and `game_engine.py` handles game logic, scoring, timing, and rendering.

`main`

`button`

The README confirms that **Task 1 is specifically the inverted outcome lookup bug**, while Tasks 2–4 are separate optional tasks that should not be touched.

`README`

What needs to change

Only `game_engine.py` needs modification.

The current dictionary has the six non-draw combinations reversed: for example, `("ROCK", "SCISSORS")` currently returns `"CPU"` even though Rock should beat Scissors.

`game_engine`

I changed **only those six values**:

- `ROCK → SCISSORS = PLAYER`
- `SCISSORS → PAPER = PLAYER`
- `PAPER → ROCK = PLAYER`
- `SCISSORS → ROCK = CPU`
- `PAPER → SCISSORS = CPU`
- `ROCK → PAPER = CPU`

Everything else—including random CPU selection, scoring, 1.8-second timing, UI, and rendering—remains unchanged. The existing scoring logic already correctly increments the player when `determine_winner()` returns `"PLAYER"` and the CPU when it returns `"CPU"`.

`game_engine`

Updated `game_engine.py`

```

import random
import pygame
from game.button import ChoiceButton

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.choices = ["ROCK", "PAPER", "SCISSORS"]
        btn_w, btn_h = 130, 50
        gap = 20
        total_w = 3 * btn_w + 2 * gap
        start_x = (width - total_w) // 2
        btn_y = height - 85

        self.buttons = [
            ChoiceButton("ROCK", pygame.Rect(start_x, btn_y, btn_w, btn_h), (160, 50),
            ChoiceButton("PAPER", pygame.Rect(start_x + btn_w + gap, btn_y, btn_w, btn_h), (160, 50),
            ChoiceButton("SCISSORS", pygame.Rect(start_x + 2 * (btn_w + gap), btn_y, btn_w, btn_h), (160, 50),
        ]

        self.player_choice = None
        self.cpu_choice = None
        self.result_text = "Make your move!"
        self.result_color = (220, 225, 235)

        self.player_score = 0
        self.cpu_score = 0

        self.round_resolved_time = 0
        self.display_duration = 1800
        self.showing_result = False

        self.font_title = pygame.font.SysFont(None, 36)
        self.font_hud = pygame.font.SysFont(None, 26)
        self.font_arena = pygame.font.SysFont(None, 32)

    def determine_winner(self, player, cpu):
        if player == cpu:
            return "TIE"

        # Corrected outcome mappings:
        # Rock beats Scissors
        # Scissors beats Paper
        # Paper beats Rock
        rules = {
            ("ROCK", "SCISSORS"): "PLAYER",
            ("SCISSORS", "PAPER"): "PLAYER",

```

```

        ("PAPER", "ROCK"): "PLAYER",
        ("SCISSORS", "ROCK"): "CPU",
        ("PAPER", "SCISSORS"): "CPU",
        ("ROCK", "PAPER"): "CPU",
    }

    return rules.get((player, cpu), "TIE")

def play_round(self, choice):
    self.player_choice = choice
    self.cpu_choice = random.choice(self.choices)

    outcome = self.determine_winner(self.player_choice, self.cpu_choice)
    if outcome == "PLAYER":
        self.player_score += 1
        self.result_text = f"You Win! {self.player_choice} beats {self.cpu_choice}"
        self.result_color = (80, 230, 120)
    elif outcome == "CPU":
        self.cpu_score += 1
        self.result_text = f"You Lose! {self.cpu_choice} beats {self.player_choice}"
        self.result_color = (240, 80, 80)
    else:
        self.result_text = f"It's a Draw! Both picked {self.player_choice}."
        self.result_color = (240, 210, 80)

    self.showing_result = True
    self.round_resolved_time = pygame.time.get_ticks()

def handle_event(self, event):
    if event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
        for btn in self.buttons:
            if btn.contains(event.pos):
                self.play_round(btn.choice_name)
                break

def update(self):
    now = pygame.time.get_ticks()
    if self.showing_result and (now - self.round_resolved_time >= self.display_duration):
        self.player_choice = None
        self.cpu_choice = None
        self.result_text = "Make your move!"
        self.result_color = (190, 195, 205)
        self.showing_result = False

def render(self, screen):
    screen.fill((24, 28, 36))

    title_surf = self.font_title.render("Rock Paper Scissors", True, (245, 245, 245))
    screen.blit(title_surf, (self.width // 2 - title_surf.get_width() // 2, 14))

    p_surf = self.font_hud.render(f"Player Score: {self.player_score}", True, (190, 195, 205))
    c_surf = self.font_hud.render(f"CPU Score: {self.cpu_score}", True, (255, 120, 120))

```

```
screen.blit(p_surf, (35, 52))
screen.blit(c_surf, (self.width - c_surf.get_width() - 35, 52))

pygame.draw.line(screen, (45, 52, 66), (25, 82), (self.width - 25, 82), 2)

p_str = self.player_choice if self.player_choice else "--"
c_str = self.cpu_choice if self.cpu_choice else "--"

arena_p = self.font_arena.render(f"Your Pick: {p_str}", True, (225, 225, 230))
arena_c = self.font_arena.render(f"CPU Pick: {c_str}", True, (225, 225, 230))
screen.blit(arena_p, (self.width // 2 - arena_p.get_width() // 2, 115))
screen.blit(arena_c, (self.width // 2 - arena_c.get_width() // 2, 155))

res_surf = self.font_arena.render(self.result_text, True, self.result_color)
screen.blit(res_surf, (self.width // 2 - res_surf.get_width() // 2, 205))

for btn in self.buttons:
    btn.render(screen)
```

Manual testing

Run:

```
python main.py
```

Then verify these cases. Since the CPU chooses randomly, you may need to play multiple rounds until the required CPU choice appears.

Player	CPU	Expected result	Score
ROCK	SCISSORS	You Win!	Player +1
SCISSORS	PAPER	You Win!	Player +1
PAPER	ROCK	You Win!	Player +1
SCISSORS	ROCK	You Lose!	CPU +1
PAPER	SCISSORS	You Lose!	CPU +1
ROCK	PAPER	You Lose!	CPU +1
ROCK	ROCK	Draw	No score change
PAPER	PAPER	Draw	No score change

Player	CPU	Expected result	Score
SCISSORS	SCISSORS	Draw	No score change

The README's expected behavior also explicitly requires these three winning relationships and identical-choice draws.

README

That is the complete Task 1 change. Do not modify `main.py` or `button.py`, and do not add any Task 2, 3, or 4 functionality.

Sources



I am continuing my Vibe Coding lab. Task 1 has already been completed and tested. The current code contains the Task 1 fix.

For this prompt, work ONLY on Task 2. Do not implement Task 3 or Task 4.

Task 2:
Implement a "First to X Wins" match victory state.

Currently, the game plays indefinitely and keeps accumulating scores. Implement a target score so that once either the player or CPU reaches the target score, the game transitions to a `GAME_OVER` match victory state.

Requirements:

1. Inspect the current code and understand how scores, rounds, events, updates, and rendering are handled.
2. Introduce a clear target score for the match.
3. When either `PLAYER` or `CPU` reaches the target score, transition to a `GAME_OVER` state.
4. Display the overall match winner on the `GAME_OVER` screen.
5. Display a prompt telling the player to press `R` to restart.
6. Pressing `R` must reset both player and CPU scores and

return the game to the normal playing state.

7. Preserve the Task 1 winner evaluation logic exactly as it currently works.

8. Preserve the existing 1.8-second round-result display behavior.

9. Do not implement adaptive AI.

10. Do not implement animated hand icons or shake animations.

11. Do not unnecessarily rewrite unrelated code.

12. Keep the implementation consistent with the existing Pygame architecture.

13. Explain exactly which files/functions were changed and why.

14. Give me clear manual test cases for verifying the target score, GAME_OVER state, winner display, and R reset.

Make only the changes necessary for Task 2.

Show less ^

Worked for 7s

Since Task 1 is already complete, I'll preserve its corrected `determine_winner()` mapping exactly and make the Task 2 changes only in `game_engine.py`.

The existing architecture already centralizes scoring, event handling, updating, and rendering inside `GameEngine`, while `main.py` simply drives that engine each frame.

`main` The current scoring happens inside `play_round()`, and the 1.8-second result timer is handled by `update()`. `game_engine +1`

Implementation choice

I'll use:

Python

Run

```
self.target_score = 3
```

So the match is **First to 3 Wins**.

To preserve the existing **1.8-second round-result display**, reaching 3 points will mark the match as finished, but the current round's result remains visible for the normal 1.8

seconds. After that timer expires, the game enters the visible `GAME_OVER` state. This avoids changing the existing round timing behavior.

Only `game_engine.py` needs to change. `main.py` and `button.py` remain untouched.

Updated `game_engine.py`

Python

Run

```
import random
import pygame
from game.button import ChoiceButton

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.choices = ["ROCK", "PAPER", "SCISSORS"]
        btn_w, btn_h = 130, 50
        gap = 20
        total_w = 3 * btn_w + 2 * gap
        start_x = (width - total_w) // 2
        btn_y = height - 85

        self.buttons = [
            ChoiceButton("ROCK", pygame.Rect(start_x, btn_y, btn_w, btn_h), (160, 50),
            ChoiceButton("PAPER", pygame.Rect(start_x + btn_w + gap, btn_y, btn_w, btn_h), (160, 50),
            ChoiceButton("SCISSORS", pygame.Rect(start_x + 2 * (btn_w + gap), btn_y, btn_w, btn_h), (160, 50)
        ]

        self.player_choice = None
        self.cpu_choice = None
        self.result_text = "Make your move!"
        self.result_color = (220, 225, 235)

        self.player_score = 0
        self.cpu_score = 0

        # Task 2: First to X Wins
        self.target_score = 3
        self.game_state = "PLAYING"
        self.match_winner = None

        self.round_resolved_time = 0
        self.display_duration = 1800
        self.showing_result = False

        self.font_title = pygame.font.SysFont(None, 36)
        self.font_hud = pygame.font.SysFont(None, 26)
```



```

self.font_arena = pygame.font.SysFont(None, 32)

def determine_winner(self, player, cpu):
    if player == cpu:
        return "TIE"

    # Task 1: Corrected outcome mappings
    rules = {
        ("ROCK", "SCISSORS"): "PLAYER",
        ("SCISSORS", "PAPER"): "PLAYER",
        ("PAPER", "ROCK"): "PLAYER",
        ("SCISSORS", "ROCK"): "CPU",
        ("PAPER", "SCISSORS"): "CPU",
        ("ROCK", "PAPER"): "CPU",
    }

    return rules.get((player, cpu), "TIE")

def play_round(self, choice):
    if self.game_state == "GAME_OVER":
        return

    self.player_choice = choice
    self.cpu_choice = random.choice(self.choices)

    outcome = self.determine_winner(self.player_choice, self.cpu_choice)

    if outcome == "PLAYER":
        self.player_score += 1
        self.result_text = f"You Win! {self.player_choice} beats {self.cpu_choice}"
        self.result_color = (80, 230, 120)

        if self.player_score >= self.target_score:
            self.match_winner = "PLAYER"

    elif outcome == "CPU":
        self.cpu_score += 1
        self.result_text = f"You Lose! {self.cpu_choice} beats {self.player_choice}"
        self.result_color = (240, 80, 80)

        if self.cpu_score >= self.target_score:
            self.match_winner = "CPU"

    else:
        self.result_text = f"It's a Draw! Both picked {self.player_choice}."
        self.result_color = (240, 210, 80)

    self.showing_result = True
    self.round_resolved_time = pygame.time.get_ticks()

def handle_event(self, event):
    if event.type == pygame.KEYDOWN and event.key == pygame.K_r:

```

```

        if self.game_state == "GAME_OVER":
            self.player_score = 0
            self.cpu_score = 0
            self.player_choice = None
            self.cpu_choice = None
            self.result_text = "Make your move!"
            self.result_color = (220, 225, 235)
            self.match_winner = None
            self.showing_result = False
            self.game_state = "PLAYING"
        return

    if self.game_state == "GAME_OVER":
        return

    if event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
        for btn in self.buttons:
            if btn.contains(event.pos):
                self.play_round(btn.choice_name)
                break

    def update(self):
        now = pygame.time.get_ticks()

        if self.showing_result and (now - self.round_resolved_time >= self.display_time):
            self.player_choice = None
            self.cpu_choice = None
            self.showing_result = False

            if self.match_winner == "PLAYER":
                self.game_state = "GAME_OVER"
            elif self.match_winner == "CPU":
                self.game_state = "GAME_OVER"
            else:
                self.result_text = "Make your move!"
                self.result_color = (190, 195, 205)

    def render(self, screen):
        screen.fill((24, 28, 36))

        title_surf = self.font_title.render("Rock Paper Scissors", True, (245, 245, 245))
        screen.blit(title_surf, (self.width // 2 - title_surf.get_width() // 2, 14))

        p_surf = self.font_hud.render(
            f"Player Score: {self.player_score}",
            True,
            (100, 180, 255)
        )
        c_surf = self.font_hud.render(
            f"CPU Score: {self.cpu_score}",
            True,
            (255, 120, 120)

```

```
)

screen.blit(p_surf, (35, 52))
screen.blit(c_surf, (self.width - c_surf.get_width() - 35, 52))

pygame.draw.line(
    screen,
    (45, 52, 66),
    (25, 82),
    (self.width - 25, 82),
    2
)

if self.game_state == "GAME_OVER":
    winner_text = (
        "You Win the Match!"
        if self.match_winner == "PLAYER"
        else "CPU Wins the Match!"
    )

    winner_surf = self.font_arena.render(
        winner_text,
        True,
        (80, 230, 120) if self.match_winner == "PLAYER" else (240, 80, 80)
    )

    restart_surf = self.font_arena.render(
        "Press R to Restart",
        True,
        (225, 225, 230)
    )

    screen.blit(
        winner_surf,
        (
            self.width // 2 - winner_surf.get_width() // 2,
            125
        )
    )

    screen.blit(
        restart_surf,
        (
            self.width // 2 - restart_surf.get_width() // 2,
            175
        )
    )

    return

p_str = self.player_choice if self.player_choice else "--"
c_str = self.cpu_choice if self.cpu_choice else "--"
```

```

arena_p = self.font_arena.render(
    f"Your Pick: {p_str}",
    True,
    (225, 225, 230)
)
arena_c = self.font_arena.render(
    f"CPU Pick: {c_str}",
    True,
    (225, 225, 230)
)

screen.blit(
    arena_p,
    (
        self.width // 2 - arena_p.get_width() // 2,
        115
    )
)
screen.blit(
    arena_c,
    (
        self.width // 2 - arena_c.get_width() // 2,
        155
    )
)

res_surf = self.font_arena.render(
    self.result_text,
    True,
    self.result_color
)

screen.blit(
    res_surf,
    (
        self.width // 2 - res_surf.get_width() // 2,
        205
    )
)

for btn in self.buttons:
    btn.render(screen)

```

Exactly what changed

1. Added the target score

Inside `__init__()`:

Python

Run

```
self.target_score = 3
```

This makes the match **First to 3 Wins**.

2. Added match state

=

Python



...

```
self.game_state = "PLAYING"
self.match_winner = None
```

There are now two relevant states:

- "PLAYING" — normal gameplay
- "GAME_OVER" — match has ended

3. Updated play_round()

The existing Task 1 evaluation remains **exactly the same**.

After scoring, we check:

Python

Run

```
if self.player_score >= self.target_score:
    self.match_winner = "PLAYER"
```

or:

Python

Run

```
if self.cpu_score >= self.target_score:
    self.match_winner = "CPU"
```

No changes were made to CPU selection. It still uses:

Python

Run

```
random.choice(self.choices)
```

so **Task 3 is not implemented**.

4. Updated update()

The existing 1800 ms timer remains:

Python

Run

```
now - self.round_resolved_time >= self.display_duration
```

The round result therefore still remains visible for **1.8 seconds**.

After that timer finishes, if somebody reached the target, we transition to:

Python

Run

```
self.game_state = "GAME_OVER"
```

5. Updated `handle_event()`

When the game is over, pressing **R** performs the reset:

Python

Run

```
self.player_score = 0  
self.cpu_score = 0
```

and returns to:

Python

Run

```
self.game_state = "PLAYING"
```

Other keys and mouse clicks do nothing while the match is over.

6. Updated `render()`

When:

Python

Run

```
self.game_state == "GAME_OVER"
```

the game displays either:

You Win the Match!

or:

CPU Wins the Match!

and:

Press R to Restart

The existing scoreboard is still shown.

No icons, animations, or adaptive AI were added.

Manual testing

Run:

```
Bash
```

```
python main.py
```

Test 1 — Target score

The target is 3.

Play rounds until either side reaches 3.

Expected:

```
Player Score: 3
```

or:

```
CPU Score: 3
```

The final round's normal result should remain visible for approximately **1.8 seconds**.

Then the game should show the GAME_OVER screen.

Test 2 — Player wins the match

Try to make the player reach 3 wins.

For example, if the CPU randomly gives you:

```
ROCK vs SCISSORS  
SCISSORS vs PAPER  
PAPER vs ROCK
```

the player should reach:

Player Score: 3

CPU Score: 0

After the final round's 1.8-second result display:

You Win the Match!

Press R to Restart

should appear.

Test 3 — CPU wins the match

Allow the CPU to reach 3.

For example:

ROCK vs PAPER

PAPER vs SCISSORS

SCISSORS vs ROCK

Expected:

CPU Score: 3

followed by:

CPU Wins the Match!

Press R to Restart

Test 4 — GAME_OVER prevents further rounds

Once the GAME_OVER screen appears:

1. Click ROCK.
2. Click PAPER.
3. Click SCISSORS.

Nothing should happen.

The scores must remain unchanged.

Test 5 — R resets the match

While the GAME_OVER screen is displayed, press:

R

Expected:

Player Score: 0

CPU Score: 0

The GAME_OVER screen disappears and the normal:

Make your move!

screen returns.

You should then be able to play a new match normally.

Test 6 — Task 1 still works

Before finishing, verify the six non-draw combinations still behave correctly:

Player	CPU	Expected
ROCK	SCISSORS	Player wins
SCISSORS	PAPER	Player wins
PAPER	ROCK	Player wins
ROCK	PAPER	CPU wins
PAPER	SCISSORS	CPU wins
SCISSORS	ROCK	CPU wins

And:

Player	CPU	Expected
ROCK	ROCK	Draw
PAPER	PAPER	Draw

Player	CPU	Expected
SCISSORS	SCISSORS	Draw

The README explicitly defines these as the expected Task 1 rules.

README +1

Files changed: `game_engine.py` only.

Files unchanged: `main.py` , `button.py` .

Task 3 and Task 4 are not implemented.

Sources



Today 2:46 PM

I am continuing my Vibe Coding lab. Tasks 1 and 2 have already been completed and tested. The current code contains both previous task implementations.

For this prompt, work ONLY on Task 3. Do not implement Task 4.

Task 3:

Implement a dynamic/adaptive CPU counter-strategy based on the player's move history.

Currently, the CPU chooses its move completely randomly using `random.choice(self.choices)`. Replace this with an adaptive strategy.

Requirements:

1. Inspect the current code and understand how player choices and CPU choices are handled.
2. Track the player's choice history during the current match.
3. Determine which move the player has favored based on their previous choices.
4. Bias the CPU toward selecting the counter-move to the player's most frequently used move.
5. Retain some randomness so the CPU does not always choose the same counter-move.

6. Handle the early-game situation when there is little or no player history appropriately.
7. Ensure the strategy works correctly for ROCK, PAPER, and SCISSORS.
8. Preserve the Task 1 winner evaluation logic exactly as it currently works.
9. Preserve the Task 2 target-score, GAME_OVER, winner display, and R reset behavior exactly as it currently works.
10. Do not implement Task 4 or add animated icons/shake animations.
11. Do not unnecessarily rewrite unrelated parts of the project.
12. Keep the implementation consistent with the existing Pygame/object-oriented architecture.
13. Explain exactly which files/functions you changed and why.
14. Explain the adaptive strategy in simple terms.
15. Give me clear manual test cases that I can use to verify that the CPU adapts to repeated player choices.
16. Provide the complete updated code for every file that needs to change.

Make only the changes necessary for Task 3.

Show less ^

Yes. Since Tasks 1 and 2 are already working, **Task 3 only needs changes to** `game_engine.py` . I'll keep the Task 1 mappings and Task 2 match-state/reset logic unchanged, and only replace the CPU's random-selection mechanism with an adaptive one.

Adaptive strategy in simple terms

The CPU will:

1. Keep a history of every move the player has made in the current match.
2. Find the player's **most frequently used move**.
3. Determine the move that beats it:
 - Player favors **ROCK** → CPU favors **PAPER**
 - Player favors **PAPER** → CPU favors **SCISSORS**
 - Player favors **SCISSORS** → CPU favors **ROCK**
4. Choose that counter-move **70% of the time**.

5. Choose randomly among all three moves the other **30% of the time**, so the CPU isn't completely predictable.
6. If there is no history yet, the CPU behaves randomly as before.
7. Pressing **R** for Task 2 resets the player history as well, because it starts a new match.

This is a relatively small change: **one history variable, one helper function, one change to `play_round()`, and one reset addition.**

Complete updated `game_engine.py`

Python

Run

```
import random
import pygame
from game.button import ChoiceButton

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.choices = ["ROCK", "PAPER", "SCISSORS"]
        btn_w, btn_h = 130, 50
        gap = 20
        total_w = 3 * btn_w + 2 * gap
        start_x = (width - total_w) // 2
        btn_y = height - 85

        self.buttons = [
            ChoiceButton("ROCK", pygame.Rect(start_x, btn_y, btn_w, btn_h), (160, 50),
            ChoiceButton("PAPER", pygame.Rect(start_x + btn_w + gap, btn_y, btn_w, btn_h), (160, 50),
            ChoiceButton("SCISSORS", pygame.Rect(start_x + 2 * (btn_w + gap), btn_y, btn_w, btn_h), (160, 50)
        ]

        self.player_choice = None
        self.cpu_choice = None
        self.result_text = "Make your move!"
        self.result_color = (220, 225, 235)

        self.player_score = 0
        self.cpu_score = 0

        # Task 2: First to X Wins
        self.target_score = 3
        self.game_state = "PLAYING"
        self.match_winner = None

        # Task 3: Track player choices during the current match
```

```

self.player_history = []

self.round_resolved_time = 0
self.display_duration = 1800
self.showing_result = False

self.font_title = pygame.font.SysFont(None, 36)
self.font_hud = pygame.font.SysFont(None, 26)
self.font_arena = pygame.font.SysFont(None, 32)

def determine_winner(self, player, cpu):
    if player == cpu:
        return "TIE"

    # Task 1: Corrected outcome mappings
    rules = {
        ("ROCK", "SCISSORS"): "PLAYER",
        ("SCISSORS", "PAPER"): "PLAYER",
        ("PAPER", "ROCK"): "PLAYER",
        ("SCISSORS", "ROCK"): "CPU",
        ("PAPER", "SCISSORS"): "CPU",
        ("ROCK", "PAPER"): "CPU",
    }

    return rules.get((player, cpu), "TIE")

def choose_cpu_move(self):
    """
    Task 3: Choose the CPU move based on the player's history.

    If there is no history yet, choose randomly.

    Once there is history:
    - Find the player's most frequently used move.
    - 70% of the time, choose the move that counters it.
    - 30% of the time, choose a random move.
    """

    if not self.player_history:
        return random.choice(self.choices)

    # Count how often the player has used each move.
    move_counts = {
        choice: self.player_history.count(choice)
        for choice in self.choices
    }

    # Find the player's most frequently used move.
    most_frequent_move = max(
        move_counts,
        key=move_counts.get
    )

```

```
# Find the move that beats the player's most frequent move.
counter_moves = {
    "ROCK": "PAPER",
    "PAPER": "SCISSORS",
    "SCISSORS": "ROCK",
}

counter_move = counter_moves[most_frequent_move]

# Keep some randomness so the CPU is not completely predictable.
if random.random() < 0.7:
    return counter_move

return random.choice(self.choices)

def play_round(self, choice):
    if self.game_state == "GAME_OVER":
        return

    self.player_choice = choice

    # Task 3: Record the player's move before selecting
    # the CPU's response.
    self.player_history.append(choice)

    self.cpu_choice = self.choose_cpu_move()

    outcome = self.determine_winner(
        self.player_choice,
        self.cpu_choice
    )

    if outcome == "PLAYER":
        self.player_score += 1
        self.result_text = (
            f"You Win! {self.player_choice} beats {self.cpu_choice}."
        )
        self.result_color = (80, 230, 120)

        if self.player_score >= self.target_score:
            self.match_winner = "PLAYER"

    elif outcome == "CPU":
        self.cpu_score += 1
        self.result_text = (
            f"You Lose! {self.cpu_choice} beats {self.player_choice}."
        )
        self.result_color = (240, 80, 80)

        if self.cpu_score >= self.target_score:
            self.match_winner = "CPU"
```

```
    else:
        self.result_text = (
            f"It's a Draw! Both picked {self.player_choice}."
        )
        self.result_color = (240, 210, 80)

    self.showing_result = True
    self.round_resolved_time = pygame.time.get_ticks()

def handle_event(self, event):
    if event.type == pygame.KEYDOWN and event.key == pygame.K_r:
        if self.game_state == "GAME_OVER":
            self.player_score = 0
            self.cpu_score = 0
            self.player_choice = None
            self.cpu_choice = None
            self.result_text = "Make your move!"
            self.result_color = (220, 225, 235)
            self.match_winner = None

            # Task 3: Start a fresh player history
            self.player_history = []

            self.showing_result = False
            self.game_state = "PLAYING"

        return

    if self.game_state == "GAME_OVER":
        return

    if event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
        for btn in self.buttons:
            if btn.contains(event.pos):
                self.play_round(btn.choice_name)
                break

def update(self):
    now = pygame.time.get_ticks()

    if self.showing_result and (
        now - self.round_resolved_time >= self.display_duration
    ):
        self.player_choice = None
        self.cpu_choice = None
        self.showing_result = False

        if self.match_winner == "PLAYER":
            self.game_state = "GAME_OVER"
        elif self.match_winner == "CPU":
            self.game_state = "GAME_OVER"
```

```
        else:
            self.result_text = "Make your move!"
            self.result_color = (190, 195, 205)

    def render(self, screen):
        screen.fill((24, 28, 36))

        title_surf = self.font_title.render(
            "Rock Paper Scissors",
            True,
            (245, 245, 245)
        )
        screen.blit(
            title_surf,
            (
                self.width // 2 - title_surf.get_width() // 2,
                14
            )
        )

        p_surf = self.font_hud.render(
            f"Player Score: {self.player_score}",
            True,
            (100, 180, 255)
        )
        c_surf = self.font_hud.render(
            f"CPU Score: {self.cpu_score}",
            True,
            (255, 120, 120)
        )

        screen.blit(p_surf, (35, 52))
        screen.blit(
            c_surf,
            (
                self.width - c_surf.get_width() - 35,
                52
            )
        )

        pygame.draw.line(
            screen,
            (45, 52, 66),
            (25, 82),
            (self.width - 25, 82),
            2
        )

        if self.game_state == "GAME_OVER":
            winner_text = (
                "You Win the Match!"
                if self.match_winner == "PLAYER"
```



```

        else "CPU Wins the Match!"
    )

    winner_surf = self.font_arena.render(
        winner_text,
        True,
        (
            (80, 230, 120)
            if self.match_winner == "PLAYER"
            else (240, 80, 80)
        )
    )

    restart_surf = self.font_arena.render(
        "Press R to Restart",
        True,
        (225, 225, 230)
    )

    screen.blit(
        winner_surf,
        (
            self.width // 2 - winner_surf.get_width() // 2,
            125
        )
    )

    screen.blit(
        restart_surf,
        (
            self.width // 2 - restart_surf.get_width() // 2,
            175
        )
    )

    return

p_str = self.player_choice if self.player_choice else "--"
c_str = self.cpu_choice if self.cpu_choice else "--"

arena_p = self.font_arena.render(
    f"Your Pick: {p_str}",
    True,
    (225, 225, 230)
)

arena_c = self.font_arena.render(
    f"CPU Pick: {c_str}",
    True,
    (225, 225, 230)
)

screen.blit(

```

```

        arena_p,
        (
            self.width // 2 - arena_p.get_width() // 2,
            115
        )
    )
    screen.blit(
        arena_c,
        (
            self.width // 2 - arena_c.get_width() // 2,
            155
        )
    )

    res_surf = self.font_arena.render(
        self.result_text,
        True,
        self.result_color
    )

    screen.blit(
        res_surf,
        (
            self.width // 2 - res_surf.get_width() // 2,
            205
        )
    )

    for btn in self.buttons:
        btn.render(screen)

```

Exactly what changed

Only `game_engine.py` needs to change.

1. Added player history

In `__init__()`:

Python

```
self.player_history = []
```

Run

This stores moves such as:

```
["ROCK", "ROCK", "PAPER", "ROCK"]
```

The history belongs to the **current match**.

2. Added `choose_cpu_move()`

This is the main Task 3 addition.

The CPU first checks:

Python

Run

```
if not self.player_history:  
    return random.choice(self.choices)
```

So the first round is still completely random because the CPU has no information about the player yet.

After that, it counts the moves:

Python

Run

```
move_counts = {  
    choice: self.player_history.count(choice)  
    for choice in self.choices  
}
```

For example:

```
ROCK      → 5  
PAPER     → 1  
SCISSORS  → 2
```

The CPU identifies ROCK as the player's most frequent move.

Then it uses:

Python

Run

```
counter_moves = {  
    "ROCK": "PAPER",  
    "PAPER": "SCISSORS",  
    "SCISSORS": "ROCK",  
}
```

Therefore, if ROCK is the player's favorite, PAPER becomes the CPU's preferred response.

3. Added 70/30 randomness

The CPU doesn't always use the counter:

Python

Run

```

if random.random() < 0.7:
    return counter_move

return random.choice(self.choices)

```

So approximately:

- 70% → counter the player's most frequent move
- 30% → completely random choice

This makes the CPU adaptive without making its behavior completely predictable.

4. Player history is updated in `play_round()`

The player's move is recorded:

Python

Run

```
self.player_history.append(choice)
```

before the CPU selects its move.

So the CPU can adapt based on previous choices.

5. History is reset with Task 2's R

When the match ends and the player presses R :

Python

Run

```
self.player_history = []
```

This is important because a new match should not inherit the previous match's behavior.

6. Task 1 is preserved

The existing `determine_winner()` mappings remain unchanged:

Python

Run

```

("ROCK", "SCISSORS"): "PLAYER",
("SCISSORS", "PAPER"): "PLAYER",
("PAPER", "ROCK"): "PLAYER",

```

and:

Python

Run

```

("SCISSORS", "ROCK"): "CPU",
("PAPER", "SCISSORS"): "CPU",

```

```
("ROCK", "PAPER"): "CPU",
```

So Task 1 is untouched.

7. Task 2 is preserved

The following are also retained:

- `target_score = 3`
- `PLAYING / GAME_OVER`
- Match winner detection
- 1.8-second result timer
- `GAME_OVER` screen
- Press R to Restart
- Score reset
- Match reset

The README describes the original game's timed result behavior as 1.8 seconds and the Task 2 requirement as a target-score `GAME_OVER` state. [README +1](#)

Manual testing for Task 3

Because the CPU is intentionally random **30% of the time**, don't expect it to counter every single round. You're testing a **bias**, not a guaranteed response.

Test 1 — Repeated ROCK

Start a new match.

Play:

```
ROCK
ROCK
ROCK
ROCK
ROCK
```

After the first few rounds, the player's history will heavily favor ROCK.

The CPU's counter is:

```
ROCK → PAPER
```

Therefore, you should see **PAPER from the CPU frequently**.

You may occasionally see ROCK or SCISSORS because of the 30% randomness.

Test 2 — Repeated PAPER

Restart the match with R .

Then repeatedly choose:

PAPER
PAPER
PAPER
PAPER
PAPER

The CPU should increasingly favor:

SCISSORS

because:

PAPER → SCISSORS

Again, occasional other CPU choices are expected.

Test 3 — Repeated SCISSORS

Restart and repeatedly choose:

SCISSORS
SCISSORS
SCISSORS
SCISSORS
SCISSORS

The CPU should frequently choose:

ROCK

because:

SCISSORS → ROCK

Test 4 — Early game

Immediately after pressing R , play one round.

There is no previous history, so the CPU should choose randomly.

The important thing is that the CPU **does not attempt to counter a nonexistent history**.

Test 5 — Mixed history

Try:

ROCK
PAPER
ROCK
ROCK
SCISSORS
ROCK

The history is:

ROCK = 4
PAPER = 1
SCISSORS = 1

ROCK is therefore the most frequent move.

The CPU should consequently favor:

PAPER

Test 6 — Verify Task 2 still works

Continue playing until either score reaches 3.

Verify:

1. Final round result appears.
2. It remains visible for approximately **1.8 seconds**.
3. GAME_OVER screen appears.
4. Further mouse clicks don't start rounds.
5. Press R .
6. Both scores become 0 .
7. The game returns to normal PLAYING state.

8. The CPU starts fresh without using the previous match's history.

Files changed

Changed:

- `game_engine.py`

Not changed:

- `main.py`
- `button.py`

Not implemented:

- Task 4 animated icons
- Shake animation
- Any other unrelated features

So this remains a focused **Task 3-only modification** on top of your completed Tasks 1 and 2.

Sources



I am continuing my Vibe Coding lab. Tasks 1, 2, and 3 have already been completed and tested successfully.

For this prompt, work **ONLY** on Task 4. Do not change the behavior or logic of Tasks 1, 2, or 3.

Task 4:

Enhance `game_engine.render()` so that the player's and CPU's selections are displayed using procedural geometric icons instead of only plain text strings.

Requirements:

1. Create a procedural geometric representation for ROCK, such as a stone/rock shape.
2. Create a procedural geometric representation for PAPER, such as a sheet/paper outline.
3. Create a procedural geometric representation for SCISSORS, such as two crossing blades.

4. Display the appropriate icon for both the player's and CPU's selected moves.
5. Add a brief shake animation before revealing the selections.
6. Preserve the existing 1.8-second round-result display behavior.
7. Preserve the Task 1 winner evaluation logic exactly as it currently works.
8. Preserve the Task 2 first-to-3 scoring, GAME_OVER state, winner display, and R reset behavior.
9. Preserve the Task 3 adaptive CPU behavior exactly as it currently works.
10. Do not modify the winner calculation or adaptive AI algorithm.
11. Do not modify the target score.
12. Do not add external image assets; use Pygame drawing primitives such as polygons, lines, circles, rectangles, and similar primitives.
13. Integrate the animation into the existing Pygame architecture rather than rewriting the project.
14. Keep the implementation readable and reasonably simple.
15. The icons should only appear when selections are being displayed; the normal game UI should continue to work when there are no selections.
16. The shake animation should be brief and should settle before the final result is displayed.
17. Make sure mouse buttons remain functional after the animation/result finishes.
18. Make sure GAME_OVER and R reset continue to work correctly.
19. Explain exactly which functions/files were changed.
20. Provide the complete updated code for every file that needs modification.
21. Give clear manual test cases for verifying the icons, shake animation, existing scoring, adaptive AI, and GAME_OVER behavior.

Make only the changes necessary for Task 4.

Show less ^

Worked for 7s

Since Tasks 1–3 are already working, Task 4 can be isolated to the **rendering/animation layer**. I'll leave the winner logic, target score, match state, reset behavior, and adaptive CPU selection untouched.

Only `game_engine.py` needs modification; `main.py` and `button.py` remain unchanged. The existing architecture already calls `engine.update()` and `engine.render(screen)` every frame, so the animation can be driven by the existing `pygame.time.get_ticks()` timing without changing the game loop. main

What Task 4 adds

- **ROCK:** irregular polygon/stone shape
- **PAPER:** outlined sheet with folded corner
- **SCISSORS:** two crossing blades with handles
- A **350 ms reveal/shake phase** at the beginning of each round
- After 350 ms, the icons settle and the normal result is displayed.
- The overall existing **1800 ms round timer is unchanged**.
- Once the 1800 ms expires, the existing Task 2 logic clears the selections or enters `GAME_OVER`.
- Mouse input and adaptive AI are untouched.

The key idea is that `round_resolved_time` is already recorded when a round starts, so `render()` can calculate how long the current reveal has been running. The existing 1800 ms timer in `update()` remains unchanged. game_engine +1

Complete updated `game_engine.py`

Python

Run

```
import random
import pygame
from game.button import ChoiceButton

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.choices = ["ROCK", "PAPER", "SCISSORS"]
        btn_w, btn_h = 130, 50
        gap = 20
        total_w = 3 * btn_w + 2 * gap
        start_x = (width - total_w) // 2
        btn_y = height - 85

        self.buttons = [
```

```

        ChoiceButton(
            "ROCK",
            pygame.Rect(start_x, btn_y, btn_w, btn_h),
            (160, 50, 50),
            (200, 70, 70)
        ),
        ChoiceButton(
            "PAPER",
            pygame.Rect(
                start_x + btn_w + gap,
                btn_y,
                btn_w,
                btn_h
            ),
            (40, 100, 170),
            (60, 130, 210)
        ),
        ChoiceButton(
            "SCISSORS",
            pygame.Rect(
                start_x + 2 * (btn_w + gap),
                btn_y,
                btn_w,
                btn_h
            ),
            (180, 140, 30),
            (220, 180, 50)
        ),
    ]

    self.player_choice = None
    self.cpu_choice = None
    self.result_text = "Make your move!"
    self.result_color = (220, 225, 235)

    self.player_score = 0
    self.cpu_score = 0

    # Task 2: First to X Wins
    self.target_score = 3
    self.game_state = "PLAYING"
    self.match_winner = None

    # Task 3: Track player choices during the current match
    self.player_history = []

    self.round_resolved_time = 0
    self.display_duration = 1800
    self.showing_result = False

    # Task 4: Duration of the icon reveal/shake animation
    self.shake_duration = 350

```

```

self.font_title = pygame.font.SysFont(None, 36)
self.font_hud = pygame.font.SysFont(None, 26)
self.font_arena = pygame.font.SysFont(None, 32)

def determine_winner(self, player, cpu):
    if player == cpu:
        return "TIE"

    # Task 1: Corrected outcome mappings
    rules = {
        ("ROCK", "SCISSORS"): "PLAYER",
        ("SCISSORS", "PAPER"): "PLAYER",
        ("PAPER", "ROCK"): "PLAYER",
        ("SCISSORS", "ROCK"): "CPU",
        ("PAPER", "SCISSORS"): "CPU",
        ("ROCK", "PAPER"): "CPU",
    }

    return rules.get((player, cpu), "TIE")

def choose_cpu_move(self):
    """
    Task 3: Choose the CPU move based on the player's history.

    If there is no history yet, choose randomly.

    Once there is history:
    - Find the player's most frequently used move.
    - 70% of the time, choose the move that counters it.
    - 30% of the time, choose a random move.
    """

    if not self.player_history:
        return random.choice(self.choices)

    move_counts = {
        choice: self.player_history.count(choice)
        for choice in self.choices
    }

    most_frequent_move = max(
        move_counts,
        key=move_counts.get
    )

    counter_moves = {
        "ROCK": "PAPER",
        "PAPER": "SCISSORS",
        "SCISSORS": "ROCK",
    }

```

```
counter_move = counter_moves[most_frequent_move]

if random.random() < 0.7:
    return counter_move

return random.choice(self.choices)

def play_round(self, choice):
    if self.game_state == "GAME_OVER":
        return

    self.player_choice = choice

    # Task 3: Record the player's move
    self.player_history.append(choice)

    self.cpu_choice = self.choose_cpu_move()

    outcome = self.determine_winner(
        self.player_choice,
        self.cpu_choice
    )

    if outcome == "PLAYER":
        self.player_score += 1
        self.result_text = (
            f"You Win! {self.player_choice} beats {self.cpu_choice}."
        )
        self.result_color = (80, 230, 120)

        if self.player_score >= self.target_score:
            self.match_winner = "PLAYER"

    elif outcome == "CPU":
        self.cpu_score += 1
        self.result_text = (
            f"You Lose! {self.cpu_choice} beats {self.player_choice}."
        )
        self.result_color = (240, 80, 80)

        if self.cpu_score >= self.target_score:
            self.match_winner = "CPU"

    else:
        self.result_text = (
            f"It's a Draw! Both picked {self.player_choice}."
        )
        self.result_color = (240, 210, 80)

    self.showing_result = True
    self.round_resolved_time = pygame.time.get_ticks()
```

```

def handle_event(self, event):
    if event.type == pygame.KEYDOWN and event.key == pygame.K_r:
        if self.game_state == "GAME_OVER":
            self.player_score = 0
            self.cpu_score = 0
            self.player_choice = None
            self.cpu_choice = None
            self.result_text = "Make your move!"
            self.result_color = (220, 225, 235)
            self.match_winner = None

            # Task 3: Reset player history for the new match
            self.player_history = []

            self.showing_result = False
            self.game_state = "PLAYING"

        return

    if self.game_state == "GAME_OVER":
        return

    if event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
        for btn in self.buttons:
            if btn.contains(event.pos):
                self.play_round(btn.choice_name)
                break

def update(self):
    now = pygame.time.get_ticks()

    if self.showing_result and (
        now - self.round_resolved_time >= self.display_duration
    ):
        self.player_choice = None
        self.cpu_choice = None
        self.showing_result = False

        if self.match_winner == "PLAYER":
            self.game_state = "GAME_OVER"
        elif self.match_winner == "CPU":
            self.game_state = "GAME_OVER"
        else:
            self.result_text = "Make your move!"
            self.result_color = (190, 195, 205)

# -----
# Task 4: Procedural geometric icons
# -----

def draw_rock_icon(self, screen, center):
    """Draw a simple procedural rock/stone icon."""

```

```
cx, cy = center

points = [
    (cx - 42, cy + 22),
    (cx - 36, cy - 15),
    (cx - 15, cy - 34),
    (cx + 18, cy - 31),
    (cx + 40, cy - 8),
    (cx + 34, cy + 24),
    (cx + 8, cy + 36),
    (cx - 20, cy + 32),
]

pygame.draw.polygon(
    screen,
    (120, 125, 135),
    points
)

pygame.draw.polygon(
    screen,
    (185, 188, 195),
    [
        (cx - 15, cy - 34),
        (cx + 18, cy - 31),
        (cx + 5, cy - 5),
        (cx - 22, cy - 8),
    ]
)

pygame.draw.line(
    screen,
    (70, 75, 85),
    (cx - 42, cy + 22),
    (cx - 20, cy + 32),
    3
)

pygame.draw.line(
    screen,
    (70, 75, 85),
    (cx - 20, cy + 32),
    (cx + 8, cy + 36),
    3
)

pygame.draw.line(
    screen,
    (70, 75, 85),
    (cx + 8, cy + 36),
    (cx + 34, cy + 24),
    3
)
```

```
)

def draw_paper_icon(self, screen, center):
    """Draw a procedural sheet-of-paper icon."""
    cx, cy = center

    paper_rect = pygame.Rect(
        cx - 32,
        cy - 42,
        64,
        84
    )

    pygame.draw.rect(
        screen,
        (235, 235, 240),
        paper_rect,
        border_radius=3
    )

    pygame.draw.rect(
        screen,
        (90, 95, 105),
        paper_rect,
        width=3,
        border_radius=3
    )

    # Folded top-right corner
    fold = [
        (cx + 10, cy - 42),
        (cx + 32, cy - 20),
        (cx + 10, cy - 20),
    ]

    pygame.draw.polygon(
        screen,
        (190, 195, 205),
        fold
    )

    pygame.draw.line(
        screen,
        (90, 95, 105),
        (cx + 10, cy - 42),
        (cx + 10, cy - 20),
        3
    )

    pygame.draw.line(
        screen,
        (90, 95, 105),
```



```

        (cx + 10, cy - 20),
        (cx + 32, cy - 20),
        3
    )

# Simple paper lines
for offset in (-5, 8, 21):
    pygame.draw.line(
        screen,
        (150, 155, 165),
        (cx - 20, cy + offset),
        (cx + 20, cy + offset),
        2
    )

def draw_scissors_icon(self, screen, center):
    """Draw procedural crossing scissors blades and handles."""
    cx, cy = center

    # Handles
    pygame.draw.circle(
        screen,
        (220, 150, 45),
        (cx - 22, cy + 24),
        13,
        4
    )

    pygame.draw.circle(
        screen,
        (220, 150, 45),
        (cx + 22, cy + 24),
        13,
        4
    )

    # Crossing blades
    pygame.draw.line(
        screen,
        (190, 195, 205),
        (cx - 13, cy + 14),
        (cx + 36, cy - 35),
        9
    )

    pygame.draw.line(
        screen,
        (220, 225, 235),
        (cx + 13, cy + 14),
        (cx - 36, cy - 35),
        9
    )

```

```

# Blade edges
pygame.draw.line(
    screen,
    (85, 90, 100),
    (cx - 13, cy + 14),
    (cx + 36, cy - 35),
    2
)

pygame.draw.line(
    screen,
    (85, 90, 100),
    (cx + 13, cy + 14),
    (cx - 36, cy - 35),
    2
)

# Center pivot
pygame.draw.circle(
    screen,
    (90, 95, 105),
    (cx, cy + 8),
    5
)

def draw_choice_icon(self, screen, choice, center):
    """Draw the appropriate icon for a selected move."""
    if choice == "ROCK":
        self.draw_rock_icon(screen, center)
    elif choice == "PAPER":
        self.draw_paper_icon(screen, center)
    elif choice == "SCISSORS":
        self.draw_scissors_icon(screen, center)

def get_shake_offset(self, elapsed):
    """
    Return a small horizontal/vertical offset during the
    initial reveal animation. The offset decreases to zero.
    """
    if elapsed >= self.shake_duration:
        return 0, 0

    progress = elapsed / self.shake_duration

    # Strong at the beginning, then settles to zero.
    amplitude = int(10 * (1.0 - progress))

    if amplitude <= 0:
        return 0, 0

    # Fast alternating shake.

```

```
direction = -1 if (elapsed // 45) % 2 == 0 else 1

return direction * amplitude, direction * amplitude // 2

def render(self, screen):
    screen.fill((24, 28, 36))

    title_surf = self.font_title.render(
        "Rock Paper Scissors",
        True,
        (245, 245, 245)
    )

    screen.blit(
        title_surf,
        (
            self.width // 2 - title_surf.get_width() // 2,
            14
        )
    )

    p_surf = self.font_hud.render(
        f"Player Score: {self.player_score}",
        True,
        (100, 180, 255)
    )

    c_surf = self.font_hud.render(
        f"CPU Score: {self.cpu_score}",
        True,
        (255, 120, 120)
    )

    screen.blit(p_surf, (35, 52))
    screen.blit(
        c_surf,
        (
            self.width - c_surf.get_width() - 35,
            52
        )
    )

    pygame.draw.line(
        screen,
        (45, 52, 66),
        (25, 82),
        (self.width - 25, 82),
        2
    )

    # -----
    # Task 2: GAME_OVER screen remains unchanged in behavior
```

```

# -----
if self.game_state == "GAME_OVER":
    winner_text = (
        "You Win the Match!"
        if self.match_winner == "PLAYER"
        else "CPU Wins the Match!"
    )

    winner_surf = self.font_arena.render(
        winner_text,
        True,
        (
            (80, 230, 120)
            if self.match_winner == "PLAYER"
            else (240, 80, 80)
        )
    )

    restart_surf = self.font_arena.render(
        "Press R to Restart",
        True,
        (225, 225, 230)
    )

    screen.blit(
        winner_surf,
        (
            self.width // 2 - winner_surf.get_width() // 2,
            125
        )
    )

    screen.blit(
        restart_surf,
        (
            self.width // 2 - restart_surf.get_width() // 2,
            175
        )
    )

    return

# -----
# Task 4: Selection icons and reveal animation
# -----
if self.player_choice and self.cpu_choice:
    elapsed = pygame.time.get_ticks() - self.round_resolved_time

    player_center = (self.width // 2 - 145, 145)
    cpu_center = (self.width // 2 + 145, 145)

    if elapsed < self.shake_duration:

```

```
offset_x, offset_y = self.get_shake_offset(elapsed)

player_center = (
    player_center[0] + offset_x,
    player_center[1] + offset_y
)

cpu_center = (
    cpu_center[0] - offset_x,
    cpu_center[1] - offset_y
)

# Icons shake during the reveal phase.
self.draw_choice_icon(
    screen,
    self.player_choice,
    player_center
)

self.draw_choice_icon(
    screen,
    self.cpu_choice,
    cpu_center
)

# Keep the result hidden until the shake settles.
reveal_text = self.font_arena.render(
    "Revealing...",
    True,
    (190, 195, 205)
)

screen.blit(
    reveal_text,
    (
        self.width // 2
        - reveal_text.get_width() // 2,
        235
    )
)

else:
    # Final settled positions.
    self.draw_choice_icon(
        screen,
        self.player_choice,
        player_center
    )

    self.draw_choice_icon(
        screen,
        self.cpu_choice,
```

```
        cpu_center
    )

    player_label = self.font_hud.render(
        self.player_choice,
        True,
        (225, 225, 230)
    )

    cpu_label = self.font_hud.render(
        self.cpu_choice,
        True,
        (225, 225, 230)
    )

    screen.blit(
        player_label,
        (
            player_center[0]
            - player_label.get_width() // 2,
            195
        )
    )

    screen.blit(
        cpu_label,
        (
            cpu_center[0]
            - cpu_label.get_width() // 2,
            195
        )
    )

    # Existing round result remains visible after
    # the reveal animation has settled.
    res_surf = self.font_arena.render(
        self.result_text,
        True,
        self.result_color
    )

    screen.blit(
        res_surf,
        (
            self.width // 2
            - res_surf.get_width() // 2,
            235
        )
    )

else:
    # Normal UI when there are no selections.
```

```

res_surf = self.font_arena.render(
    self.result_text,
    True,
    self.result_color
)

screen.blit(
    res_surf,
    (
        self.width // 2
        - res_surf.get_width() // 2,
        205
    )
)

# Choice buttons remain active and rendered normally.
for btn in self.buttons:
    btn.render(screen)

```

Exactly what changed

game_engine.py

I added only Task 4 functionality:

1. self.shake_duration

Python

Run

```
self.shake_duration = 350
```

This gives each selection a **350 ms animation phase**.

The existing:

Python

Run

```
self.display_duration = 1800
```

is untouched.

So the complete round-result period is still **1.8 seconds**.

2. draw_rock_icon()

Uses `pygame.draw.polygon()` and lines to create an irregular stone shape.

No image file or external asset is used.

3. `draw_paper_icon()`

Uses:

- `pygame.draw.rect()`
- `pygame.draw.polygon()`
- `pygame.draw.line()`

to create a sheet with a folded corner and paper lines.

4. `draw_scissors_icon()`

Uses:

- circles for handles
- thick lines for blades
- a center pivot

to create crossing scissors.

5. `draw_choice_icon()`

This simply selects the appropriate geometric drawing function:

```
ROCK      → draw_rock_icon()  
PAPER     → draw_paper_icon()  
SCISSORS  → draw_scissors_icon()
```

6. `get_shake_offset()`

This calculates a small changing offset during the first 350 ms.

The icons therefore move back and forth initially and gradually settle into their final positions.

There is no `sleep()` or blocking animation, so the Pygame event loop remains responsive.

7. `render()`

The old plain-text:

Your Pick: ROCK

CPU Pick: PAPER

display has been replaced by the procedural icons when a round has selections.

The sequence is now:

Player clicks

↓

CPU chooses

↓

350 ms icon shake

↓

Icons settle

↓

Result text appears

↓

Remaining round time continues

↓

1.8 seconds total

↓

Selections cleared

This is implemented entirely inside the existing rendering architecture.

What was deliberately NOT changed

Task 1

`determine_winner()` is unchanged.

The mappings remain:

ROCK → SCISSORS = PLAYER

SCISSORS → PAPER = PLAYER

PAPER → ROCK = PLAYER

and the opposite combinations remain CPU wins.

Task 2

These remain unchanged:

Python

Run

```
self.target_score = 3
self.game_state = "PLAYING"
self.match_winner = None
```

The GAME_OVER screen and R reset behavior are also unchanged.

Task 3

choose_cpu_move() is unchanged.

The adaptive behavior remains:

No history → random

History exists:

70% → counter player's most frequent move

30% → random move

The player history is also still reset when starting a new match.

main.py

No changes.

The existing loop already calls:

Python

```
engine.update()
engine.render(screen)
```

Run

every frame, which is exactly what the non-blocking animation needs.

main

button.py

No changes.

The existing buttons continue to be rendered at the end of render(), so mouse interaction remains available after the animation finishes.

button

Manual testing

Test 1 — ROCK icon

Run:

```
Bash
```

```
python main.py
```

Click **ROCK**.

You should see:

1. Rock/stone icon appears.
2. Icon shakes briefly.
3. Icon settles.
4. CPU's corresponding icon appears as well.
5. The result is displayed.
6. After the normal round duration, selections disappear.

Test 2 — PAPER icon

Click **PAPER**.

Verify that the player selection is represented by a **sheet of paper with a folded corner**, not the old plain `PAPER` text.

Test 3 — SCISSORS icon

Click **SCISSORS**.

Verify that the icon has:

- two crossing blades
- two handles
- a center pivot

Test 4 — Shake animation

When you click any move, watch the selection area carefully.

You should see the icons move slightly for approximately **0.35 seconds**, then settle.

The animation should not freeze the window.

Test 5 — Result timing

After clicking a move:

- the reveal/shake lasts about 350 ms
- the result appears after the reveal settles
- the selections disappear according to the existing **1.8-second round timer**

There should not be a new multi-second delay added to the game.

Test 6 — Mouse buttons remain functional

After a round finishes, click another button.

Expected:

```
New round starts
→ new CPU choice
→ new icon animation
→ new result
```

So the animation must not disable normal gameplay.

Test 7 — Task 1 still works

Check the displayed result against the icons:

Player	CPU	Expected
ROCK	SCISSORS	Player wins
SCISSORS	PAPER	Player wins
PAPER	ROCK	Player wins
ROCK	PAPER	CPU wins
PAPER	SCISSORS	CPU wins
SCISSORS	ROCK	CPU wins

Draws should still produce no score increase.

Test 8 — Task 3 still adapts

Play several consecutive:

ROCK
ROCK
ROCK
ROCK

The CPU should still increasingly favor **PAPER**.

Likewise:

PAPER → CPU favors SCISSORS
SCISSORS → CPU favors ROCK

The icons are only a visual change; the underlying adaptive algorithm has not been modified.

Test 9 — GAME_OVER

Play until either side reaches **3 wins**.

Expected:

1. Final round's icons appear.
2. Final result is displayed.
3. After the existing timing completes, GAME_OVER appears.
4. Match winner is displayed.
5. Press R to Restart is displayed.
6. Clicking the choice buttons does not start another round.

Test 10 — R reset

On GAME_OVER, press **R**.

Expected:

Player Score: 0
CPU Score: 0

The normal game screen returns, and you can click the buttons again.

The new match should also start with a fresh adaptive-AI history.

Files modified: `game_engine.py` **only**.

`main.py` **and** `button.py` **remain unchanged**.

Sources



ChatGPT can make mistakes. Check important info.